



VISUAL ANALYTICS: BIG DATA LAB

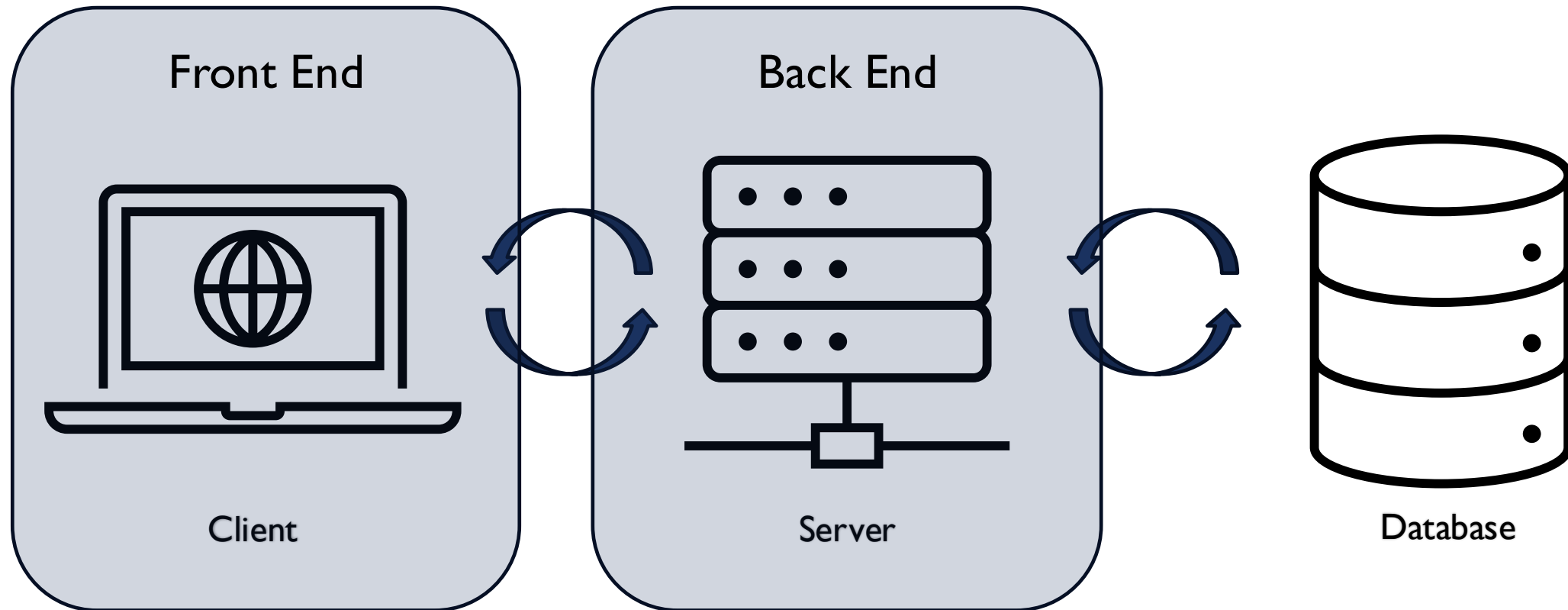
AB MOSCA (THEY/THEM)



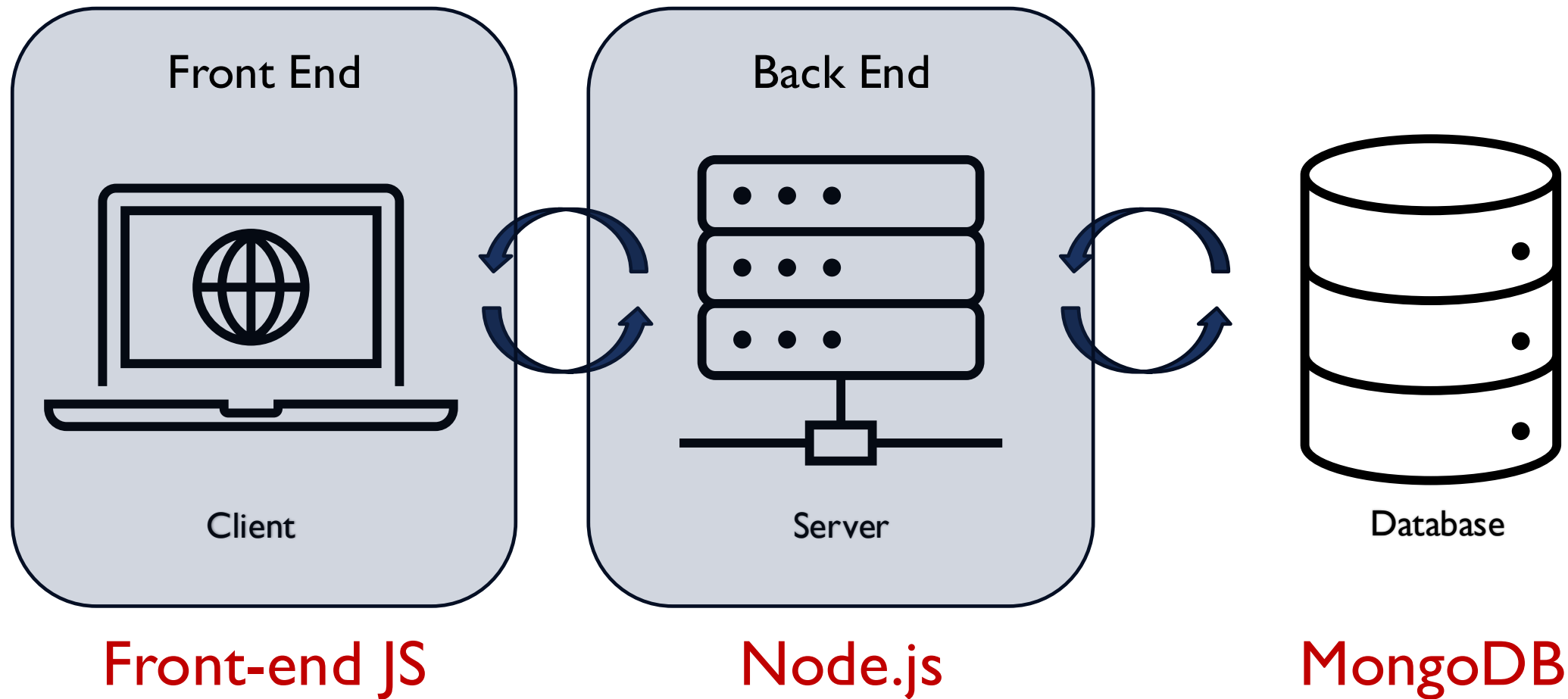
TODAY'S PLAN

- Full stack basics
- Architecture
- Required packages
- Dynamic visualization

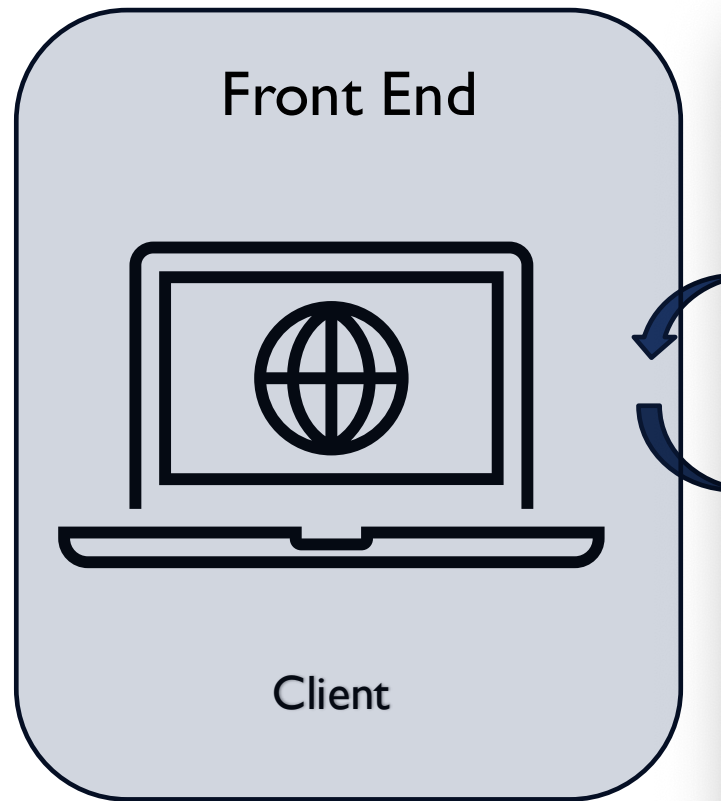
FULL STACK BASICS



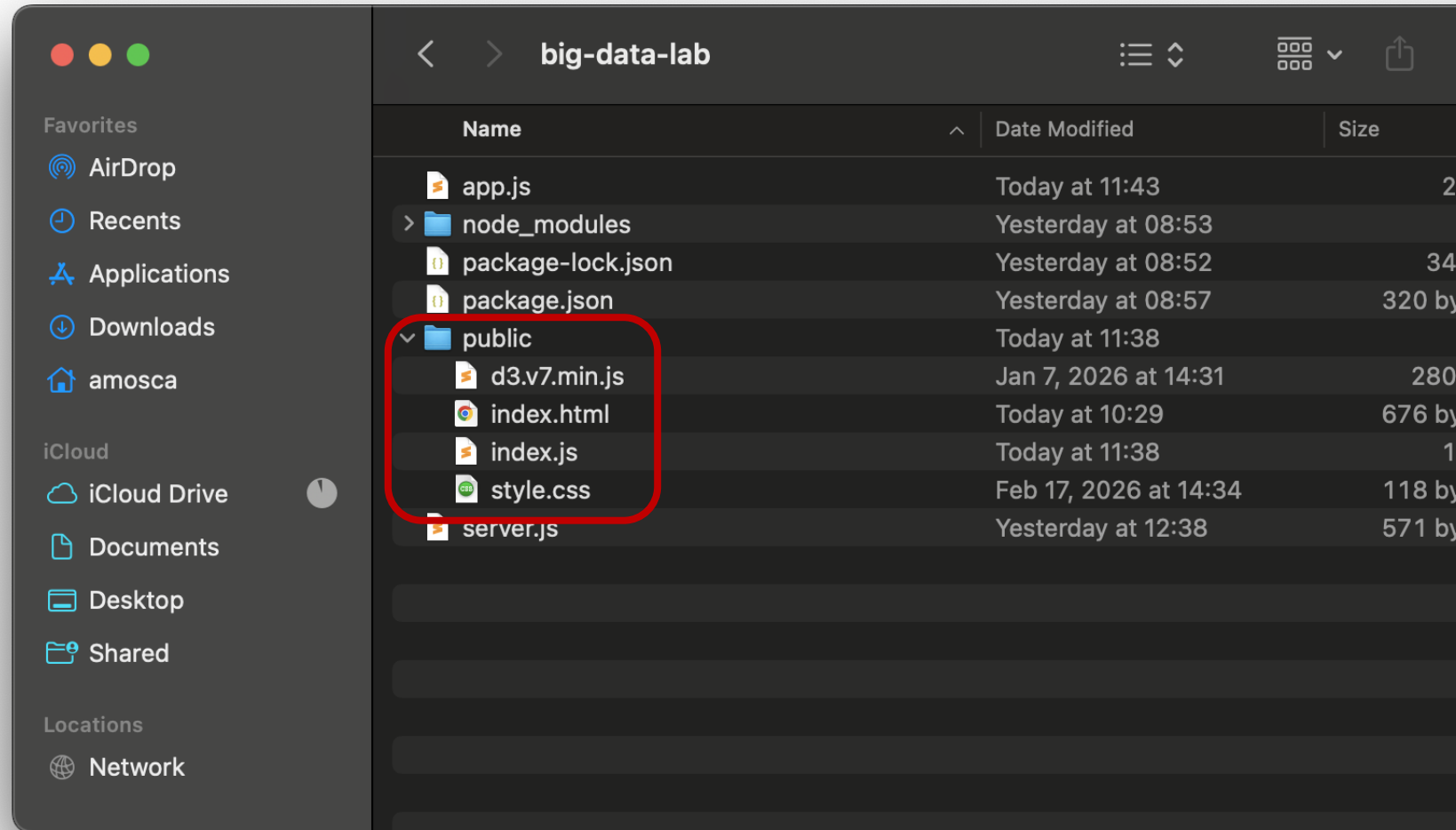
REQUIRED PACKAGES



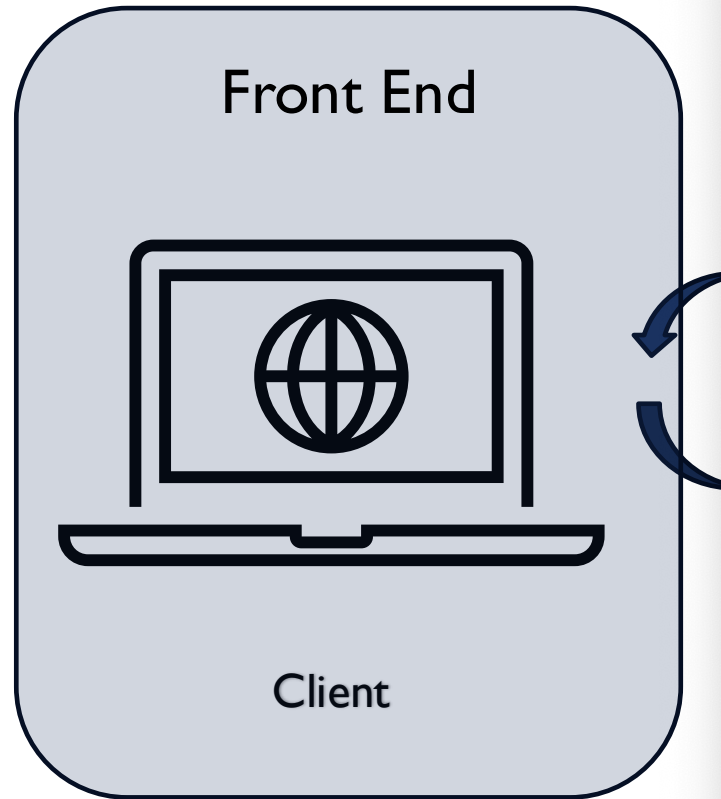
ARCHITECTURE



Front-end JS



ARCHITECTURE

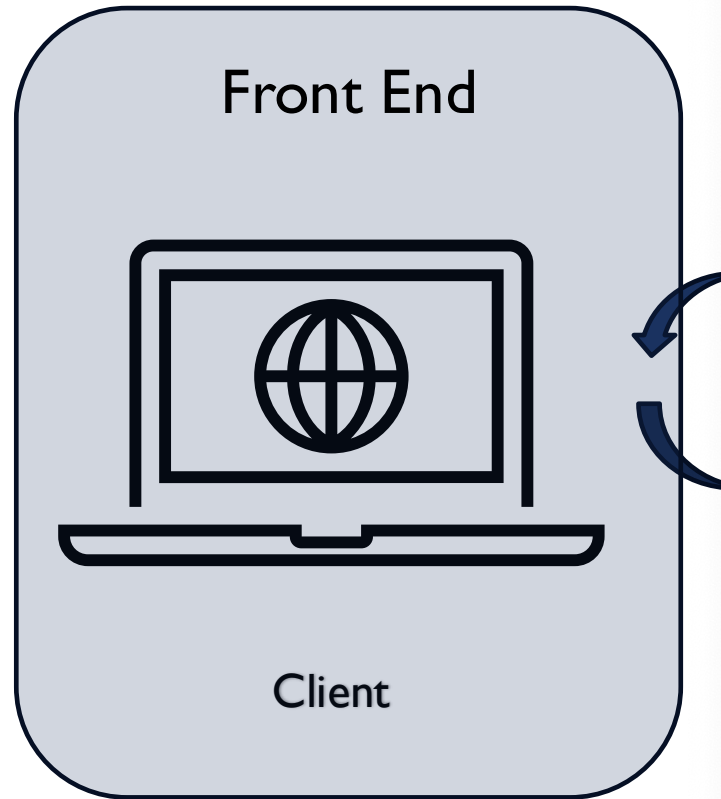


Front-end JS

```
index.html
app.js  index.js  index.html
1  <!DOCTYPE html>
2  <html>
3  <head>
4
5      <link rel="stylesheet" href="style.css">
6      <title>Big Data</title>
7      <script src="d3.v7.min.js"></script>
8
9  </head>
10 <body>
11
12     <div id="landing">
13         <h1 class="important">Big Data & D3</h1>
14     </div>
15
16     <div id="frame"></div>
17
18     <div>
19         <label for="rating">Choose a rating:</label>
20
21         <select name="rating" id="rating">
22             <option value="G">G</option>
23             <option value="PG">PG</option>
24             <option value="PG-13">PG-13</option>
25             <option value="R">R</option>
26             <option value="NR">NR</option>
27         </select>
28     </div>
29
30 <script src="index.js"></script>
31 </body>
32 </html>
33
34
```

Choose a rating:

ARCHITECTURE

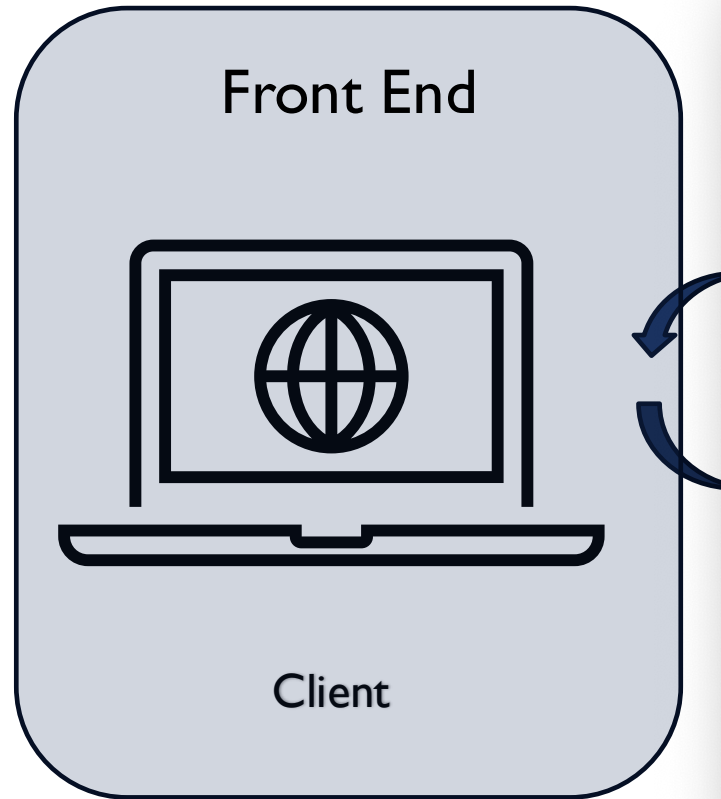


Front-end JS

```
index.js UNREGISTERED
index.js x
1
2
3 // event listener and handler for rating selection
4 const ratingSelection = d3.select("#rating");
5
6 ratingSelection.on('change', (event) => {
7   const selectedValue = event.target.value;
8   // data to send to server
9   const dataToSend = {rating: selectedValue};
10  // pass selection to server
11  fetch('http://localhost:8000/api/data', {
12    method: 'POST',
13    headers: {
14      'Content-Type': 'application/json',
15    },
16    body: JSON.stringify(dataToSend),
17  })
18  .then(response => response.json())
19  .then(data => {
20    console.log('Response from server: ', data);
21    // fetch data from server
22    fetchServerData().then((fetchResponse) => {
23      drawVis(fetchResponse.data);
24    });
25  })
26  .catch((error) => {
27    console.log('Error: ', error);
28  });
29 });
30
```

Line 33, Column 10 Spaces: 4 JavaScript

ARCHITECTURE

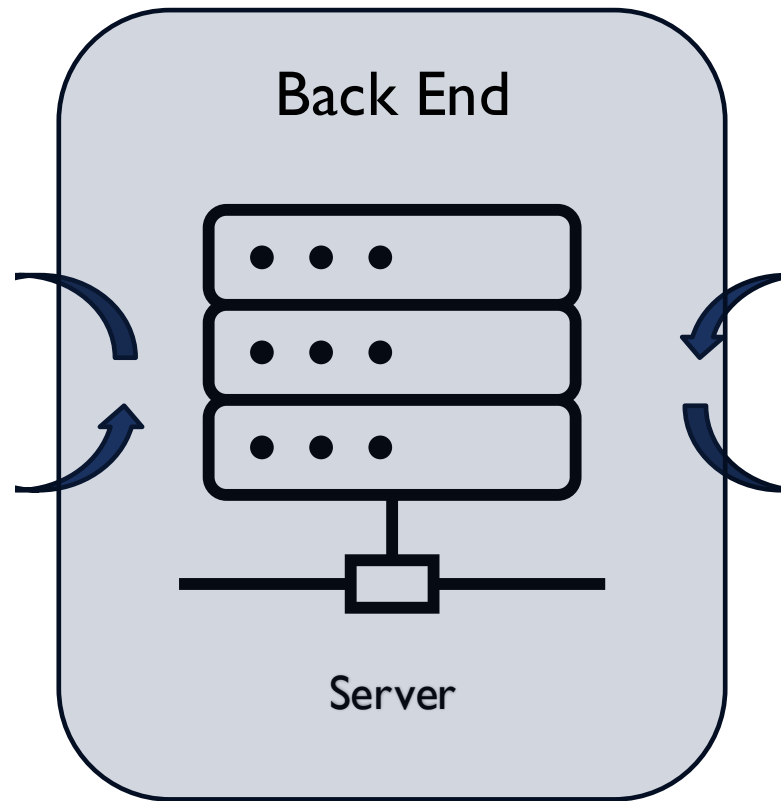
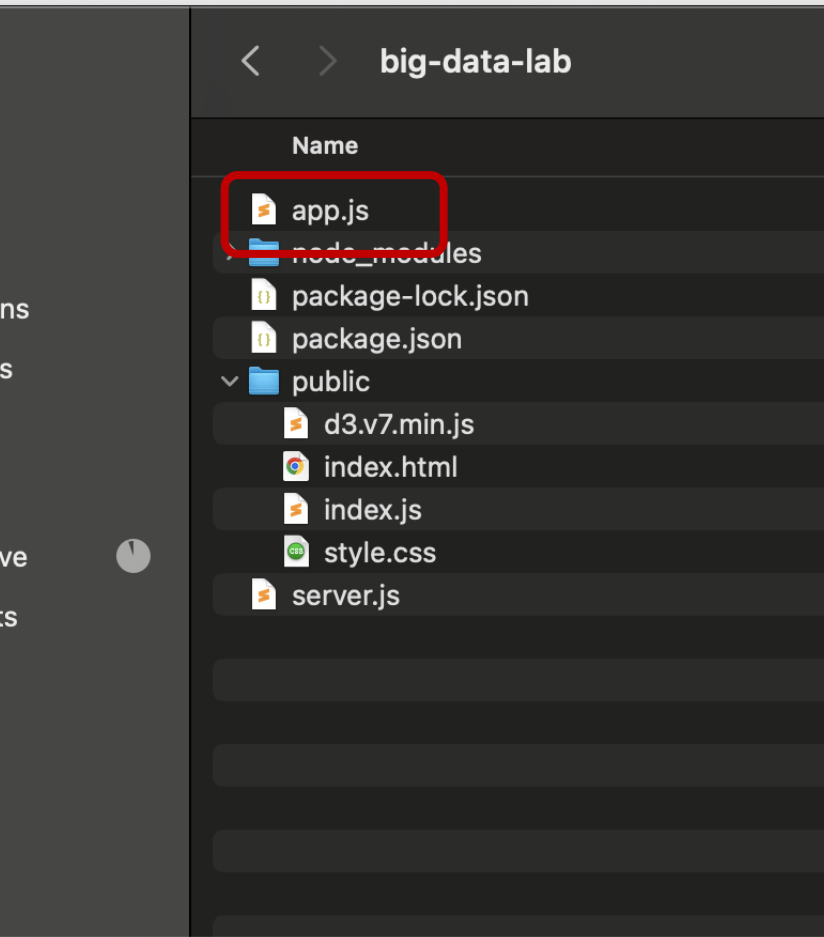


Front-end JS

```
index.js  UNREGISTERED
index.js
31 // To get data from server
32 async function fetchServerData() {
33   try {
34     const response = await fetch('/api/data');
35     if (!response.ok) {
36       throw new Error('Network response was not ok');
37     }
38     const data = await response.json();
39     console.log("Fetching server data");
40     return data;
41   } catch (error) {
42     console.error("There was a problem with your fetch: ", error);
43   }
44 }
45
46 // TODO: Build your visualization here
47 async function drawVis(myData) {
48   console.log("Data passed to drawVis(): ", myData);
49 }
50
```

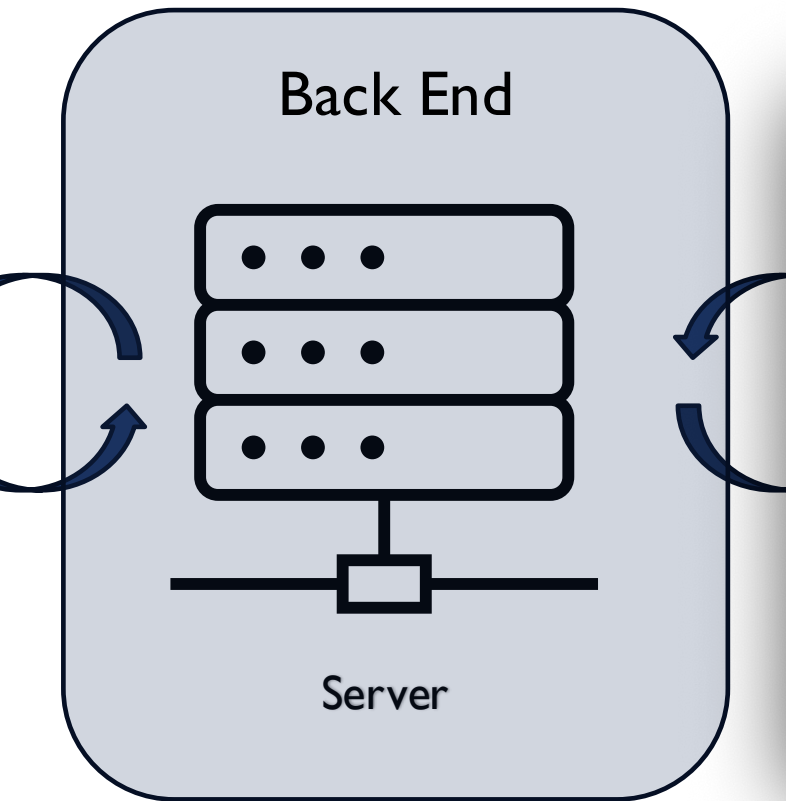
Line 50, Column 1 Spaces: 4 JavaScript

ARCHITECTURE



Node.js

ARCHITECTURE



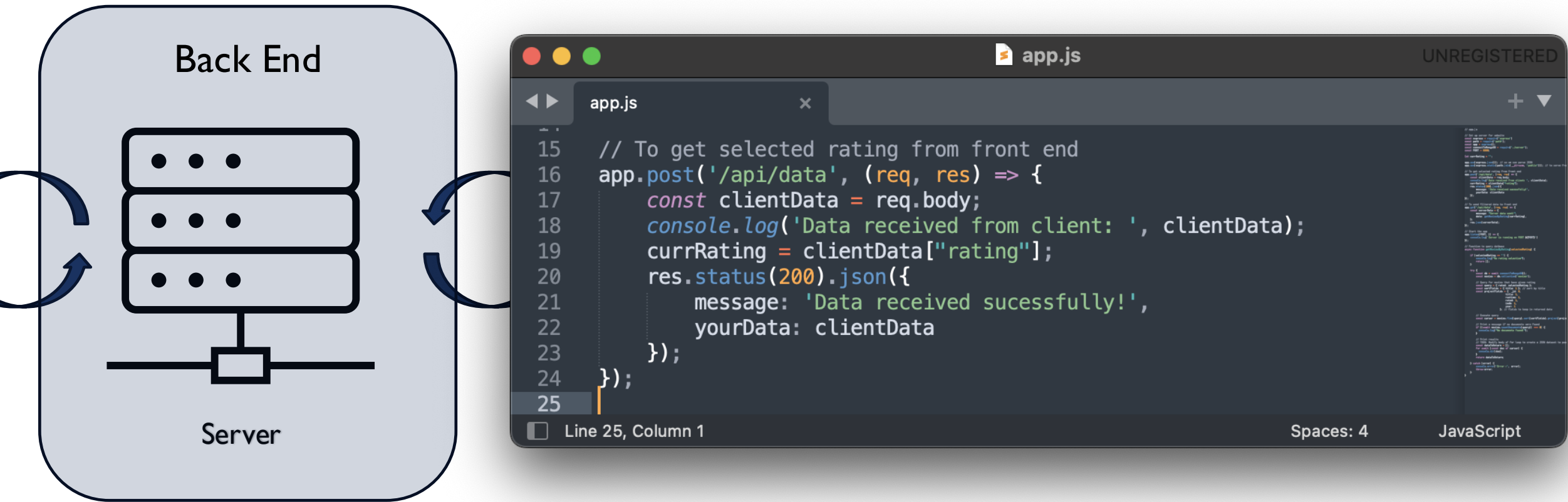
Node.js

The screenshot shows a code editor window titled "app.js". The code is as follows:

```
1 // app.js
2
3 // Set up server for website
4 const express = require('express')
5 const path = require('path');
6 const app = express();
7 const connectToMongoDB = require('./server');
8 const PORT = 8000;
9
10 let currRating = '';
11
12 app.use(express.json()); // so we can parse JSON
13 app.use(express.static(path.join(__dirname, 'public'))); // to serve from public folder
14
```

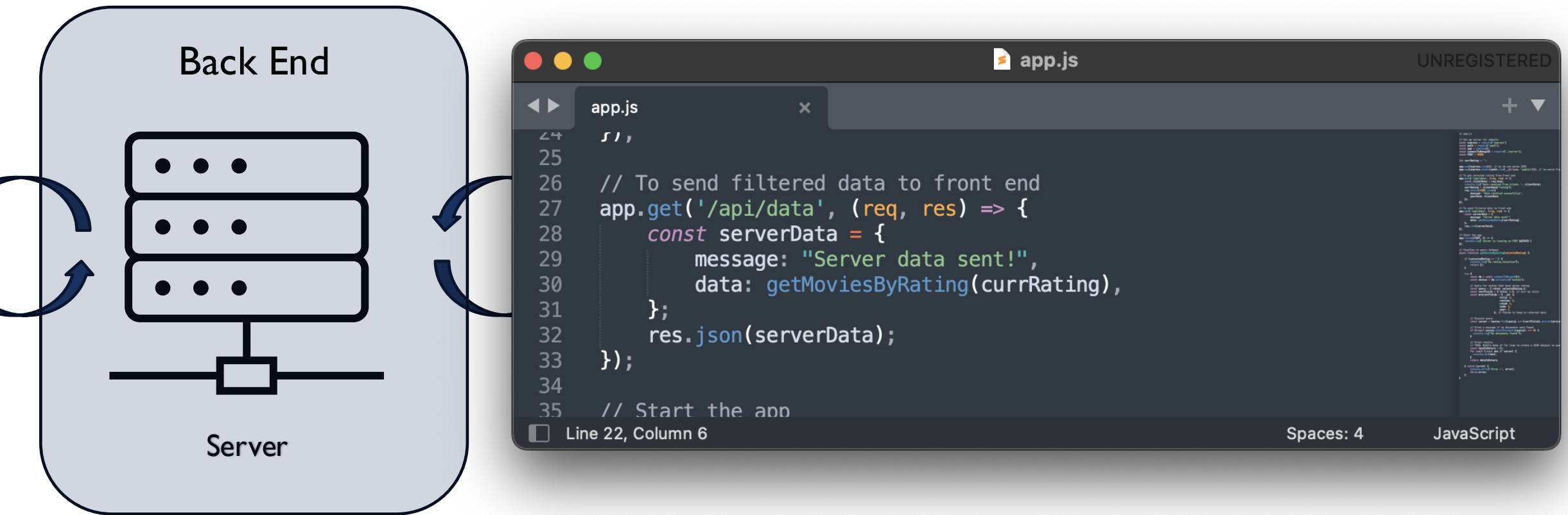
The editor interface includes a tab for "app.js", a cursor at line 1, column 1, and a status bar at the bottom right indicating "Spaces: 4".

ARCHITECTURE



Node.js

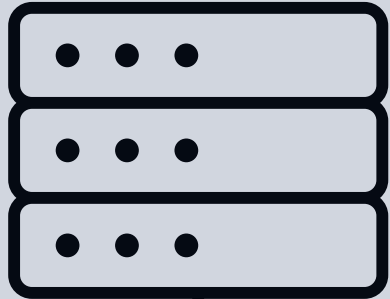
ARCHITECTURE



Node.js

ARCHITECTURE

Back End



Server

Node.js

```
app.js
// Function to query database
40 async function getMoviesByRating(selectedRating) {
41
42
43   if (selectedRating == '') {
44     console.log("No rating selection");
45     return [];
46   }
47
48   try {
49     const db = await connectToMongoDB();
50     const movies = db.collection('movies');
51
52     // Query for movies that have given rating
53     const query = { rated: selectedRating };
54     const sortFields = { title: 1 }; // sort by title
55     const projectFields = { _id: 0,
56                             title: 1,
57                             runtime: 1,
58                             rated: 1,
59                             imdb: 1,
60                             year: 1
61                           }; // fields to keep in returned data
62
63     // Execute query
64     const cursor = movies.find(query).sort(sortFields).project(projectFields);
65
66     // Print a message if no documents were found
67     if ((await movies.countDocuments(query)) === 0) {
68       console.log("No documents found!");
69     }
70
71     // Print results
72     // TODO: Modify body of for loop to create a JSON dataset to pass to the front end
73     const dataToReturn = [];
74     for await (const doc of cursor) {
75       console.dir(doc);
76     }
77     return dataToReturn;
78
79   } catch (error) {
80     console.error('Error :', error);
81     throw error;
82   }
83 }
```

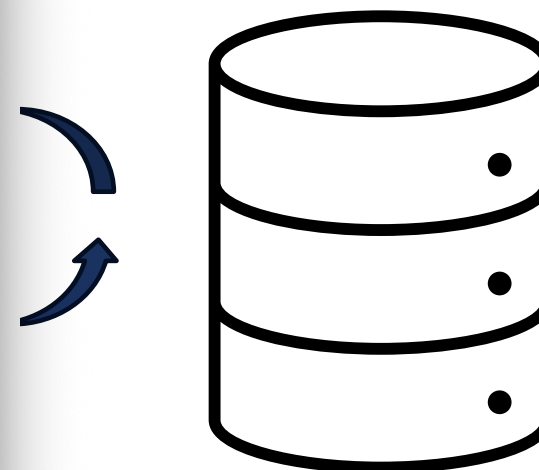
ARCHITECTURE



ARCHITECTURE

< > big-data-lab

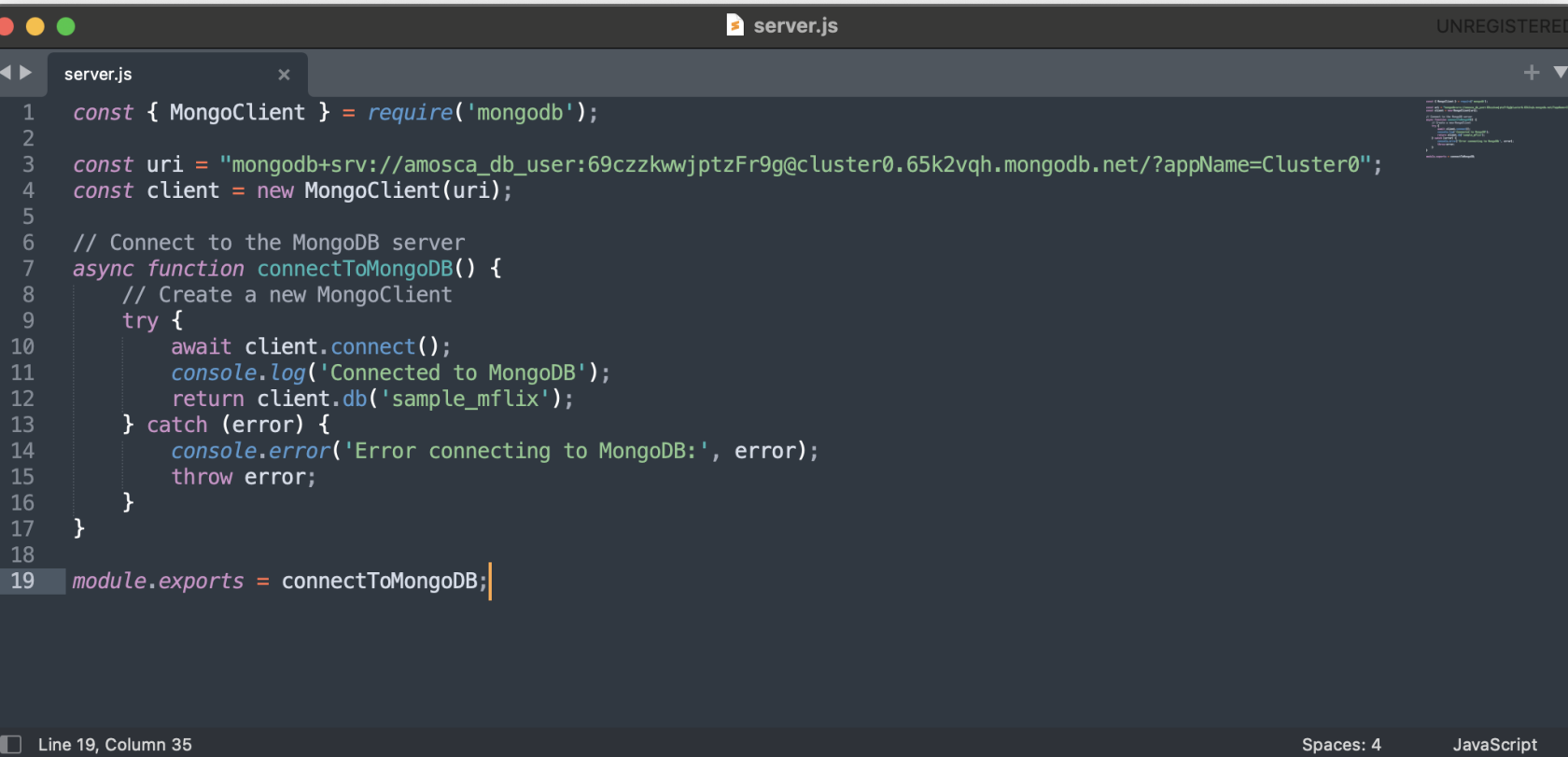
Name	Date Modified	Size	Kind
app.js	Today at 11:43	2 KB	JavaScript
> node_modules	Yesterday at 08:53	--	Folder
package-lock.json	Yesterday at 08:52	34 KB	JSON File
package.json	Yesterday at 08:57	320 bytes	JSON File
▼ public	Today at 11:38	--	Folder
d3.v7.min.js	Jan 7, 2026 at 14:31	280 KB	JavaScript
index.html	Today at 10:29	676 bytes	HTML text
index.js	Today at 11:38	1 KB	JavaScript
style.css	Feb 17, 2026 at 14:34	118 bytes	CSS File
server.js	Yesterday at 12:38	571 bytes	JavaScript



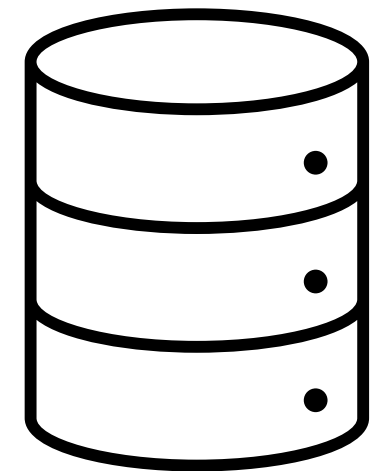
Database

MongoDB

ARCHITECTURE

A screenshot of a code editor window titled 'server.js' with a tab icon and a close button. The editor shows JavaScript code for connecting to MongoDB. The code includes a require statement for 'mongodb', a URI string, a MongoClient instance, an async function 'connectToMongoDB' with a try-catch block for connection logic, and an export statement. The status bar at the bottom indicates 'Line 19, Column 35', 'Spaces: 4', and 'JavaScript'.

```
1  const { MongoClient } = require('mongodb');
2
3  const uri = "mongodb+srv://amosca_db_user:69czzkwjptzFr9g@cluster0.65k2vqh.mongodb.net/?appName=Cluster0";
4  const client = new MongoClient(uri);
5
6  // Connect to the MongoDB server
7  async function connectToMongoDB() {
8    // Create a new MongoClient
9    try {
10     await client.connect();
11     console.log('Connected to MongoDB');
12     return client.db('sample_mflix');
13   } catch (error) {
14     console.error('Error connecting to MongoDB:', error);
15     throw error;
16   }
17 }
18
19 module.exports = connectToMongoDB;
```



Database

MongoDB

REQUIRED PACKAGES

1. Install Node.js and npm (<https://nodejs.org/en/download>)

2. Navigate to big-data-lab folder and install:

1. Express.js

```
bash
```

```
npm install express
```

2. MongoDB Node.js driver

```
bash
```

```
npm install mongodb
```

SET UP A DATABASE

1. Make a MongoDB account (<https://www.mongodb.com/>)
2. Add a cluster (choose the free option)
3. Once the cluster is created, click “Connect”
4. Select “Drivers”
5. Scroll to section 3:

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String ⓘ

☐ View full code sample

```
mongodb+srv://amosca_db_user:<db_password>@cluster0.d0cwwtk.mongodb.net/?  
appName=Cluster0
```

Replace `<db_password>` with the password for the `amosca_db_user` database user. Ensure any option params are [URL encoded](#).

6. Copy the connection string (circled above in red). Make sure `<db_password>` is a password, or reset the database user password manually and fill it in
7. Paste your connection string into `server.js`, replacing the existing string for `uri`

DYNAMIC VISUALIZATION

```
cd <path>/big-data-lab
```

```
node app.js
```

